

Дополнительная лекция по Си № 12

28 мая 2025 г.

Коновалов А.В.

Представление вещественных чисел в памяти компьютера

Типы вещественных чисел в языке Си называются `float` и `double`. Слово `float` означает *floating point*, т.е. число с плавающей точкой или плавающей запятой. Слово `double` означает число с плавающей точкой *двойной точности* (для сравнения `float` — одинарной точности). Такая вот историческая терминология. В языке Visual Basic типы вещественных чисел называются `Single` и `Double` — одинарной или двойной точности.

Язык Си также предоставляет тип `long double`, его размер и представление зависят от платформы и компилятора.

В давние времена на разных машинах использовался разный формат для представления вещественных чисел (т.е. чисел с плавающей запятой), но в середине 1980-х был предложен стандарт IEEE (институт инженеров электротехники и электроники) для представления вещественных чисел. Номер стандарта впишу в конспект потом. В дальнейшем на это стандартное представление перешли все производители процессоров.

Число типа `float` соответствует 32-разрядному двоичному стандартному представлению, типа `double` — 64-разрядному.

Тип `long double` в некоторых компиляторах (например, Microsoft Visual C++) может быть синонимом `double` (и тоже иметь размер 64 бита или 8 байт), может быть нестандартным 80-битным представлением внутри сопроцессора Intel x87, занимать 10 или 16 байт памяти (в последнем случае 6 байт не используются, размер округлён для выравнивания). Мы его рассматривать не будем.

О представлении вещественных чисел

Различают два представления: с фиксированной запятой или с плавающей запятой. В случае с фиксированной запятой, в двоичной (или десятичной) записи числа несколько разрядов отводятся на дробную часть, несколько на целую.

Сложение и вычитание таких чисел выполняется также, как и для целых. Умножение и деление требуют выполнения сдвига в результате.

Скажем, мы можем договориться использовать число `int32_t` как число с фиксированной запятой, отводя на дробную часть последние 8 бит. Диапазон представимых чисел будет от $-2^{31} / 256 = -8\,388\,608,000$ до $(2^{31} - 1) / 256 = 8\,388\,608,996$. Точность будет 8 двоичных цифр или 2–3 десятичные цифры (с точностью 0,004).

Число с плавающей запятой представляется в экспоненциальной форме: как кортеж из значения мантиссы и порядка. Т.е. число (m, e) означает $m \times b^e$, где b — выбранное основание (может быть 10 или 2).

Если мы рассматриваем привычные нам десятичные числа, то мантисса должна быть в диапазоне $1 \leq m < 10$. Т.е. число 100500 мы должны представить как $1,005 \times 10^5$ (степень буду обозначать стрелочкой) — число в нормализованной форме. Если отказаться от нормализованной формы, то у числа может быть несколько представлений: $1,005 \times 10^5$, $10,05 \times 10^4$, $0,1005 \times 10^6$ и т.д.

При использовании основания 2 мантисса будет располагаться в диапазоне $1 \leq m < 2$, т.е. мантисса, представленная как число с фиксированной точкой, всегда будет содержать (для ненулевого числа) в целой части цифру 1. А это значит, что для представления нормализованных вещественных чисел целую часть мантиссы можно не хранить (она всегда равна 1). Единица в знаке мантиссы нормализованного числа будет *подразумеваться*. Благодаря этому приёму, мы можем добавить 1 бит точности представлению мантиссы.

Заметим, что мы не можем представить 0 как нормализованное число. Для нуля используется особое представление.

Представление знаковых чисел

Двоичные числа естественным образом изображают беззнаковые числа в двоичной системе счисления. Для представления знаковых чисел требуются специальные действия. Различают три способа изображения знакового числа в двоичной форме:

- **Прямой код** — число хранится как кортеж из знакового бита и модуля (абсолютного значения) числа. Обычно знаковый бит 0 означает положительное число, 1 — отрицательное.
- **Обратный код** или дополнение до 1. Отрицательное число представляют инверсией битов положительного, старший бит используют как знаковый. Т.е., например, 8 разрядное число 40 будет представлено как 00101000, число -8 — как 11010111. Лектор об использовании таких чисел на практике не слышал, а только читал в учебниках как возможный вариант.
- **Дополнительный код** или дополнение до 2. Старший бит рассматривается как знаковый и имеет вес $-2^{(N-1)}$. Остальные биты имеют привычные положительные веса $+2^k$.

На сегодня дополнительный код повсеместно используется для представления целых знаковых чисел. Преимущество использования дополнительного кода — операции сложения и вычитания одинаково будут выполняться и для беззнаковых чисел, и для знаковых. Произведение N -разрядных чисел имеет длину $2 \times N$, при использовании дополнительного кода младшая половина произведения (т.е. младшие N разрядов) будет одинаково вычисляться и для беззнаковых чисел, и для знаковых. Это значит, что в процессоре можно обойтись одними и теми же командами для сложения, вычитания и умножения знаковых и беззнаковых чисел (что упрощает оборудование). Для деления (и остатка), всё-таки придётся реализовывать две разные команды.

Представления чисел в прямом и обратном коде имеют по два представления для нуля: $+0$ и -0 . Кроме того, в них усложняются реализации сложения и вычитания.

Представление чисел в дополнительном коде имеет свой недостаток: существует отрицательное число, у которого нет парного положительного: $100...000$, означающее $-2^{\uparrow(N-1)}$. Попытка поменять знак этого числа даст его же.

Смена знака в дополнительном коде делается путём обращения битов и прибавления единицы:

$$-x = (\sim x) + 1$$

$$-0 = -00...00 = (\sim 00...00) + 1 = 11...11 + 1 = 0$$

$$-1 = -00...001 = (\sim 00...001) + 1 = 11...110 + 1 = 11...111$$

$$-2^{\uparrow(N-1)} = -100...00 = (\sim 100...00) + 1 = 011...11 + 1 = 1000...00 = -2^{\uparrow(N-1)}$$

Для представления вещественных чисел используется прямой код: число хранится как кортеж из знакового бита и абсолютного значения (модуля).

К слову, при использовании симметричной троичной системы счисления (веса разрядов — степени тройки, цифры имеют значения -1 , 0 , $+1$), наоборот, изображение знакового числа совершенно естественно. Например, четырёхтритовое число (число из четырёх троичных цифр) будет иметь диапазон от -40 до $+40$.

Смена знака троичного числа сводится к поразрядному дополнению.

Наконец, изучаем вещественные числа

Напишем функцию, которая принимает вещественное число с плавающей запятой двойной точности, т.е. `double` и распечатывает его биты. И посмотрим на представление некоторых чисел.

```
#include <stdio.h>
#include <stdint.h>
#include <string.h>

void print_double(double num)
{
```

```
uint64_t bits;
memcpy(&bits, &num, sizeof(num));

printf("%10.5f: ", num);
for (int i = 63; i ≥ 0; --i) {
    printf("%d", (bits >> i) & 1);
}
printf("\n");
}

int main()
{
    print_double(10);
    print_double(5);
    print_double(-5);
    print_double(20);
    print_double(1.25);
    print_double(0.625);
    print_double(0.1);
    print_double(0.0);

    return 0;
}
```

Для преобразования вещественного в целое мы воспользовались функцией копирования памяти `memcpy(void *dest, void *src, size_t n)`, т.к. этот способ является переносимым и надёжным. Можно было бы воспользоваться приведением указателей, однако, Стандарт определяет т.н. `strict aliasing rules`, согласно которым, компиляторы при оптимизации могут портить семантику программ, если несколько указателей разного типа хранят один адрес. Компилятор имеет право считать, что если есть указатели разного типа, то они заведомо не ссылаются на одну и ту же память и использовать соответствующие оптимизации.

Посмотрим на результат программы:

D:\Си>a.exe

```
D:\Си>a.exe
```

[illegible]

Можно заметить, что старший бит — знаковый, хранит 0 для положительных

$$\begin{aligned} 10 &= 1010 = 1,01 \times 2^{\uparrow 3} \\ 5 &= 101 = 1,01 \times 2^{\uparrow 2} \\ 20 &= 1,01 \times 2^{\uparrow 4} \\ 1,25 &= 1,01 \times 2^{\uparrow 0} \\ 0,625 &= 1,01 \times 2^{\uparrow -1} \end{aligned}$$

Порядок у вещественных чисел хранится как положительное число в смещённом виде: т.е. к порядку прибавляется смещение. Если мы посмотрим на число 1,25, у которого двоичный порядок должен быть 0, его порядок записан как 0111111111 — 11 цифрами, представляющими число $2^{10-1} = 1023$.

Порядок, состоящий из одних двоичных единиц 11...11 тоже зарезервирован: с нулевой мантиссой он изображает ∞ (со знаком, соответственно, $+\infty$ или $-\infty$), с ненулевой мантиссой — т.н. *нечисло* (*not-a-number*, NaN) — особое значение, используемое для представления некорректного результата (например, корня из отрицательного числа).

Наибольшее нормализованное значение типа `double` представляется числом с порядком 11...10 и мантиссой из одних единиц. Чтобы не считать вручную, сформируем это число искусственно и посмотрим:

[illegible]

5

```
big_bits = 0x7FF0000000000000;
memcpy(&big, &big_bits, sizeof(big));
print_double(big);

big_bits = 0x7FF0000000050000;
memcpy(&big, &big_bits, sizeof(big));
print_double(big);
```

[illegible]

Вещественные числа также могут хранить и *денормализованные* значения: их порядок будет состоять из нулей, мантисса хранит значение с явной единицей. Однако, такие числа могут поддерживаться не всеми процессорами и количество верных знаков тем меньше, чем меньше само число.

```
uint64_t small_bits = 0x00100000000500000;  
double small;  
memcpy(&small, &small_bits, sizeof(small));  
print_double(small);  
  
small_bits = 0x00000000000500000;  
memcpy(&small, &small_bits, sizeof(small));  
print_double(small);
```

[illegible]

```
small_bits = 0x00000000000000000000000000000001;
memcpy(&small, &small_bits, sizeof(small));
print double(small);
```

6

